

计算机体系结构_p0

• 肖老师复习课知识点：

• 第一章中性能的度量——latency和throughput、CPI

- 定义
 - latency是单个指令从开始到完成的时间
 - throughput是指单位时间内完成的指令数
 - CPI是指执行一条指令平均所需的时钟周期数
- 结合实际进行分析
- 概念上要理解准确

• 流水线级数越多不一定越好？（cost）

正面影响：流水线的级数越多，意味着流水线被切得越细，每一级流水线内容纳硬件逻辑越少，意味着运行更高的主频。主频越高，流水线的吞吐率越高。

负面影响：

(1) 每一级流水线都由寄存器组成，更多的流水线级数意味着消耗更多的寄存器，产生更大的面积开销，功耗也会增大。

(2) 每一级流水线需要握手，流水线的最后一级反压信号可能会一直串扰到最前一级造成严重的时序问题。

(3) 流水线的取指阶段得知条件跳转的结果是到底跳还是不跳，因此只能进行预测，而到了流水线的末端才能通过实际运算得知该分支是真的该跳还是不该跳。如果发现真实的结果与预期的结果不一致，意味着预测失败，需要将所有预取指令全部丢失。流水线越深，则意味着浪费和损失越严重；流水线越浅，则浪费和损失越少。

- 流水线级数越多，在处理多指令的时候确实会更高效，但必然以高成本为代价，同时也会出现很多相关的副作用
- 流水线的级数受限于流水线的延迟和流水线的额外开销
- 有硬件开销
- 流水线优势是怎么提升的latency还是throughput
 - 流水线并不能减少latency
 - 流水线缩短了在一个时钟周期内给的那个信号必须通过的通路长度，增加了数据吞吐量
 - 流水线增加了指令吞吐量（单位时间内完成的指令数），但不会减少那个指令单执行时间
- 流水线的优化技术有哪些

technique	reduces
forwarding/ bypassing	potential data-hazard stalls
delayed branches	control-hazard stalls
basic dynamic scheduling (scoreboarding)	data-hazard stalls from true dependencies
dynamic scheduling with register renaming	data-hazard, anti-dep. & output dep. stalls
dynamic branch prediction	control stalls
issuing multiple instruction per clock cycle	ideal CPI
speculation	data-hazard and control-hazard stalls
dynamic memory disambiguation	data-hazard stalls with memory
loop unrolling	control hazard stalls
basic compiler pipeline scheduling	data-hazard stalls
compiler dependency analysis	ideal CPI, data-hazard stalls
software pipelining & trace scheduling	ideal CPI, data-hazard stalls
compiler speculation	ideal CPI, data-hazard stalls

- 两个局部性原理

- 概念

- 时间局部性

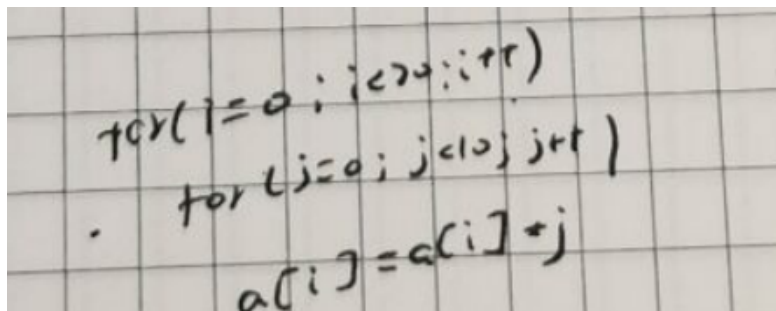
- 如果一个信息项正在被访问，那么近期他很可能还会被访问
 - e.g 程序循环， 栈

- 空间局部性

- 如果一个存储器的位置被引用，那么将来它附近的位置也会被引用

- 例子 (结合cache和循环)

- 两层循环——时间和空间局部性



Example

```
for ( i= 0 ; i< 20 ; i++)
    for( j=0; j<10; j++)
        a[i]=a[i]*j;
```

- Spatial locality: for the outer loop, it accesses memory $a[0]$, $a[1]$, ..., $a[20]$ sequentially.
- Temporal locality: for the inner loop, it accesses memory $a[i]$ repeatedly in each iteration.

- 空间局部性——外层循环

- 当访问数组第一个元素时，会把第一个元素后的若干元素一块放入缓存行，这样顺序访问数组元素时就会在缓存中命中，因而就不会去主内存读取了,后续访问也是这样

- 时间局部性—— $a[i]=a[i]*j$
 - 时间局部性:对于内部循环,它在每次迭代中重复访问内存 $a[i]$ 。
- **cache**既体现空间局部性,有体现时间局部性
 - 原因:
 - 空间局部性——块多字
 - 当发生缓存丢失时,将根据空间局部原则从主存中检索固定大小的连续存储单元块
 - 时间局部性——放到cache里
 - directly exploits temporal locality providing faster access to a smaller subset of the main memory which contains copy of data recently used (直接利用时间局部性,为包含最近使用的数据副本的更小的主内存子集提供更快的访问)

• 通用寄存器, R-TO-R

- 典型指令
 - R,J,I
- 仅sw和lw可以从memory存取数据
- 通用寄存器

- 通用寄存器三种架构
 - 只有显式的操作数,无论是寄存器还是内存位置
 - Register-register
 - 仅通过load和store指令访问内存
 - register-memory
 - 任一指令都可访问内存

• memory-memory

- 只有内存

• 优缺点对比

Type	Advantages	Disadvantages
Register-register (R,R)	Simple, fixed-length instructions encoding simple code-generation models.	Higher instruction count. Some instructions are short and bit encoding may be wasteful.
Register-memory (R,M)	Instructions can be accessed without loading the instruction format fields to be only in encode and decode great benefit.	A larger operand is displayed. Checks per instruction varies by operand location.
Memory-memory (M,M)	Program becomes more compact for waste of registers for temporaries.	Large variation in instruction sizes and in work per instruction. Memory accesses create memory bottlenecks.

Type	Advantages	Disadvantages
Register-register (R,R)	简单,指令长度固定,编码简单,代码生成模型简单。	更高的指令数。有些指令较短,而且位编码可能浪费。
Register-memory (R,M)	可以在不加载指令的情况下访问数据,指令中仅包含寄存器编号和内存地址,指令格式简单。	需要作数据搬运。每个指令的特定操作数位置的不同而不同。
Memory-memory (M,M)	程序变得紧凑,不需要寄存器。	指令大小和每指令的工作量差异很大。内存访问会造成内存瓶颈。

Example (0,3): ADD R1,R2,R3
(1,2): ADD R1,X
(3,3): ADD X1,X2,X3

$R[R1] \leftarrow R[R2] + R[R3]$
 $R[R1] \leftarrow R[R1] + R[X]$
 $R[X1] \leftarrow R[X2] + R[X3]$

Type	No. of memory addresses	Maximum No. of operands allowed	Examples
(R,R)	0	2	SPARC, MIPS, PowerPC, ALP64
(R,M)	1	2	Intel x86, Motorola 68000
(M,M)	3	0	VAX

• 分支预测器

- 1个bit到n个bit的分支预测器不是越高越好(2bit只关注自己前一个;后面再加,没有太大帮助)

• 数据冒险

- 哪几种,能自己举例子,能从代码分析出有哪种冒险,不是所有数据依赖都会造成冒险,数据依赖不管会不会有冒险都要列出来

- RAW
- WAW
- WAR
- 例题
- cache
 - 优化技术
 - 写直达
 - 写回
 - 写分配
 - 组相连
 - 减小缺失率、减小缺失代价——怎么实现的
- 循环展开——一定要掌握
 - 基本步骤
 - 怎么优化
 - 每一步如何展开
 - 时钟周期数怎么算的?
 - 展开几次
- 动态指令调度技术
 - 有哪些技术, 分别有什么优缺点
- scoreboard和tomasulo的对比

	Tomasulo's Algorithm	Scoreboard
introduced with	IBM 360/91 in 1967	CDC 6600 in 1964
resources	3 adders, 2 mul/div, 6 load, 3 store	7 int units, 4 FP units, 5 mem. reference
max window size	14	5
structural hazards	stall issuing	stall issuing
WAR hazards	avoided by register renaming	stall committing
WAW hazards	avoided by register renaming	stall issuing
control	based on reservation stations	scoreboard module
communication	broadcasting to reservation stations (and registers)	buses to/from registers

• With scoreboard both operands are read at once when they are both ready in the register file, while Tomasulo's algorithm reads each operand asap

- 不同点
 - tomasulo通过寄存器换名过程可以消除WAR和WAW竞争
 - scoreboard 能检测WAR和WAW竞争, 一旦检测到竞争, 通过插入停顿周期来解决这一竞争——记分牌不能消除WAR和WAW竞争
 - 检测竞争和控制指令执行方式不同
 - Tomasulo方法检测竞争和控制指令执行两方面功能是通过分布在每一功能单元的保留站来进行的, 因此Tomasulo方法是一种分布式方法。

- 记分牌方法的上述功能是通过统一的记分牌来实现的，因此记分牌方法是一种集中式方法
 - 写结果的方法不同
 - Tomasulo方法通过CDB直接将功能单元输出的结果送往需要该结果的所有保留站，而不必经过寄存器这一中间环节。
 - 记分牌方法是将结果写入FP寄存器,因而可能造成等待这一结果的指令都出现停顿现象，之后，所有相关指令的功能单元在读FP 寄存器时又可能出现竞争现象。
 - Tomasulo方法中无检查WAW和WAR竞争功能，因为在指令发射过程中，由 issue logic结合保留站完成了register operands的改名过程，即消除了这两存在结构相关和WAW相关时无法发射后续指令，导致整个流水线空闲；没有转发操作；种竞争。
 - CDB起到广播结果的作用，不必通过register file直接结果送到所有需要该结果的保留站和buffers.
 - Load和Store buffers相当于基本功能单
- scoreboard的主要思想
 - 顺序发射，乱序执行乱序完成；
 - 发射前检测结构相关和WAW相关；
 - 读操作数前检测RAW相关；
 - 写结果前处理WAR相关。
 - scoreboard缺陷
 - 存在结构相关和WAW相关时无法发射后续指令，导致整个流水线空闲；
 - 没有转发操作；
 - 为什么顺序发射
 - 因为顺序发射可以知道某条指令的结果会流向哪条指令，如果乱序发射则会混淆RAW，WAR相关
 - tomasulo相较于scoreboard的优点
 - 具有分布的阻塞检测机制：如果多条指令都在等待一个结果，且指令的另一个操作数都已准备就绪，公共数据总线广播这个被等待的结果后，这些指令可以同时得到这个结果数据，并同时开始进入执行段。
 - 消除了数据的写后写和先读后写相关导致的阻塞

以上内容整理于 [幕布文档](#)

计算机体系结构_p1

第一章

功率和能耗

- 功率（静态）= 电流（静态）× 电压
 - 原因：晶体管处于截止状态时也存在泄漏电流
- 能耗（动态）= $1/2 \times \text{容性负载} \times (\text{电压})^2$
- 功率（动态）= $1/2 \times \text{容性负载} \times \text{电压}^2 \times \text{开关频率}$

晶片

事实上，由于个人移动设备越来越依赖于整体片上系统（SOC），所以集成电路成为 PMD 成本的主体部分。因此，计算机设计人员必须了解芯片的成本，才能理解计算机的成本。

尽管集成电路的成本以指数形式下降，但基本的硅制造工艺没有变化：仍然（wafer）进行测试，切割成晶片（die）进行封装（见图 1-6、图 1-7 和图 1-8）。因此，集成电路的成本为：

$$\text{集成电路的成本} = \frac{\text{晶片成本} + \text{晶片测试成本} + \text{封装与最终测试成本}}{\text{最终测试成品率}}$$

本节主要讨论晶片成本，最后总结测试和封装中的关键问题。

要学习如何预测一个晶圆上的正品芯片数目，需要首先了解一个晶圆上可以放多少片。知道了这些数据，预测成本就很简单了：

$$\text{晶片成本} = \frac{\text{晶圆成本}}{\text{每个晶圆上的晶片数} \times \text{晶片成品率}}$$

成本公式第一项中的最重要特征是它对晶片尺寸非常敏感，如下所示。

这个 300 mm 的晶圆上包含 280 个全沙桥晶片，采用 32 nm 工艺时，每个晶片尺寸为 0.7 mm × 10.5 mm。（沙桥是 Intel 在 Core i7 中所用 Nehalem 的后续产品）。当晶片大小为 1.5 cm × 1.5 cm 时，每个晶圆上的晶片数大约为 282 个（感谢 Intel 提供）

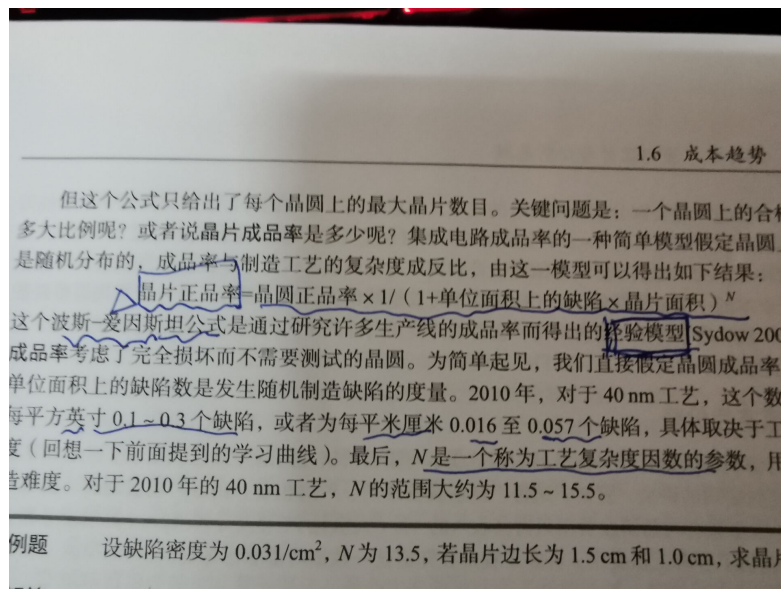
晶圆上的晶片数目大约等于晶圆面积除以晶片面积。更准确的估算公式为：

$$\text{每个晶圆上的晶片数} = \frac{\pi \times (\text{晶圆直径}/2)^2}{\text{晶片面积}} - \frac{\pi \times \text{晶圆直径}}{\sqrt{2} \times \text{晶片面积}}$$

圆面积（ πr^2 ）与晶片面积之比。第二项对“方枘圆凿”问题（也就是接口不匹配）作出补偿。将圆周（ πd ）除以方形晶圆的对角线，大约就是沿边缘排列的晶片数。

若晶片边长为 1.5 cm，求一个 300 mm（30 cm）晶圆上的晶片数目，若晶片边长为 1.0 cm，又可以有多少个晶片。

若晶片面积为 2.25 cm² 时：



• 两个关键性能指标

• Time to run the task

- execution time, response time, elapsed time, latency

• Tasks per time unit

- execution rate, bandwidth, throughput

• latency

- “real” time necessary to complete a task – important when the focus is on a single task

• 专注于单个任务是非常重要

• throughput (aka Bandwidth)

- 单位时间内完成的任务数
- 专注于多个任务时非常重要

• 性能指标 (performance metrics)

• Machine X is n times faster than machine Y

$$n = \frac{\text{executionTime}(Y)}{\text{executionTime}(X)} = \frac{\text{performance}(X)}{\text{performance}(Y)}$$

• Performance and execution time are reciprocal

- improve performance → increase performance
- improve execution time → decrease execution time

• Example

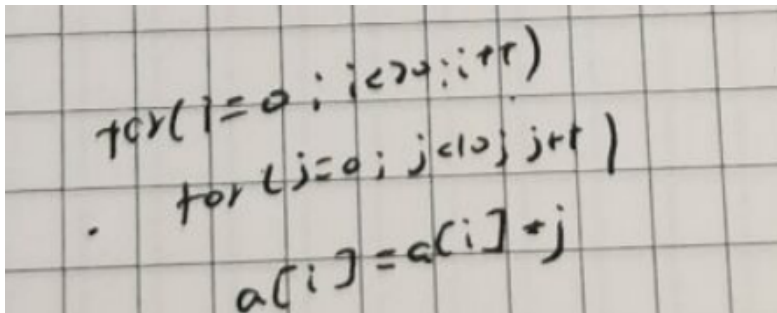
- executionTime(Y) = 4.8, executionTime(X) = 3.6
- $n = 1.33$, i.e. X is 33% faster than Y

• 局部性原理

• 时间局部性

- 如果一个信息项正在被访问，那么在近期它很可能还会被再次访问。
- 实例：程序循环、堆栈是产生局部性的原因

- 空间局部性
 - 如果一个存储器的位置被引用，那么将来它附近的位置也会被引用
- **cache**既体现空间局部性，有体现时间局部性
 - 原因：
 - 空间局部性——块多字
 - 当发生缓存丢失时，将根据空间局部原则从主存中检索固定大小的连续存储单元块
 - 时间局部性——放到cache里
 - directly exploits temporal locality providing faster access to a smaller subset of the main memory which contains copy of data recently used（直接利用时间局部性，为包含最近使用的数据副本的更小的主内存子集提供更快的访问）
- **两层循环**——时间和空间局部性



Example

```

for ( i= 0 ; i< 20 ; i++)
  for( j=0; j<10; j++)
    a[i]=a[i]*j;
  
```

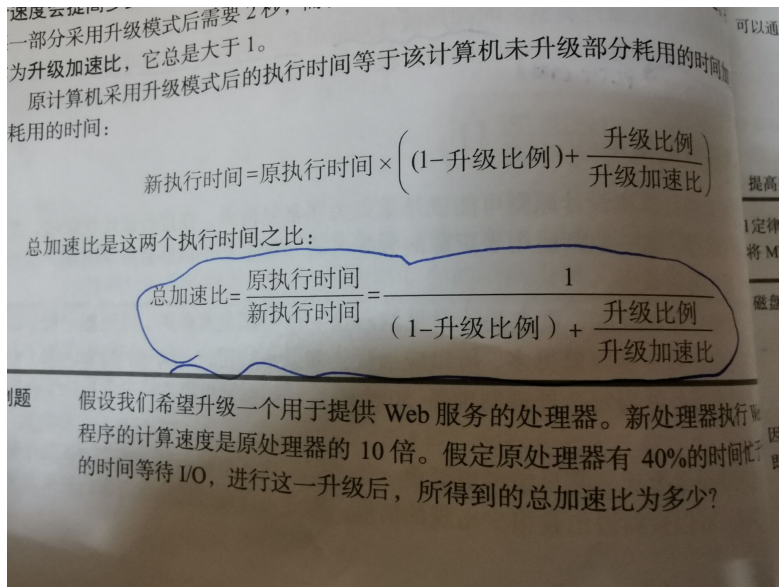
• Spatial locality: for the outer loop, it accesses memory $a[0]$, $a[1]$, ..., $a[20]$ sequentially.

• Temporal locality: for the inner loop, it accesses memory $a[i]$ repeatedly in each iteration.

- 空间局部性——外层循环
 - 当访问数组第一个元素时，会把第一个元素后的若干元素一块放入缓存行，这样顺序访问数组元素时就会在缓存中命中，因而就不会去主内存读取了,后续访问也是这样
 - 时间局部性—— $a[i]=a[i]*j$
 - 时间局部性:对于内部循环，它在每次迭代中重复访问内存 $a[i]$ 。
- 阿姆达尔定律
 - **总加速比公式**

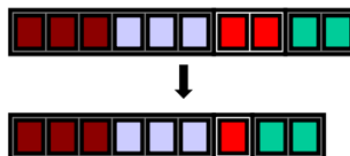
- If component x is improved by S_x and component x affects a fraction F_x of the overall execution time then

$$speedup = \frac{1}{(1 - F_x) + F_x / S_x}$$



实例

If we optimize the module for the floating-point instructions by a factor of 2, but the system will normally run programs with only 20% of floating point instructions then the speedup is only



$$speedup = \frac{1}{(1 - 0.2) + 0.2 / 2} = \frac{1}{0.9} = 1.111$$

- CPU time
 - CPI

$$CPI = \sum_i \left(\frac{IC_i}{IC} \times CPI_i \right) = \sum_i \left(IF_i \times CPI_i \right)$$

第二章

通用寄存器三种架构

- 只有显式的操作数，无论是寄存器还是内存位置
- Register-register
 - 仅通过load和store指令访问内存
- register-memory
 - 任一指令都可访问内存

- memory-memory

- 只有内存

- 优缺点对比

Type	Advantages	Disadvantages
Register-register (0,3)	Simple, fixed-length instr. encoding. Simple code generation model	Higher instruction count. Some instructions are short and bit encoding may be wasteful.
Register-memory (1,2)	Data can be accessed without loading first. Instruction format tends to be easy to encode and yields good density.	A source operand is destroyed. Clocks per instruction varies by operand location.
Memory-memory (3,3)	Program becomes most compact. No waste of registers for temporaries.	Large variation in instruction sizes and in work per instruction. Memory accesses create memory bottleneck.

Type	Advantages	Disadvantages
Register-register (0,3)	简单 指令定长 简单的代码生成模型	更高的指令数。 有些指令很短，而且位编码可能很浪费
Register-memory (1,2)	可以在不先加载的情况下访问数据。 指令格式趋向于易于编码并产生良好的密度。	源操作数被销毁。 每条指令的时钟因操作数位置的不同而不同。
Memory-memory (3,3)	程序变得最紧凑。 不需要寄存器	指令大小和每条指令的工作量差异很大 内存访问会造成内存瓶颈。

Example

(0, 3) : ADD	R1, R2, R3	$R[R1] \leftarrow R[R2] + R[R3]$
(1, 2) : ADD	R1, X	$R[R1] \leftarrow R[R1] + M[X]$
(3, 3) : ADD	X1, X2, X3	$M[X1] \leftarrow M[X2] + M[X3]$

Type (M,O)	No of memory addresses	Maximum No of operands allowed	Examples
(0, 3)	0 Register (0,3)	3	SPARC, MIPS, PowerPC, ALPHA
(1, 2)	1 Register (1,2)	2	Intel 80x86, Motorola 68000
(3, 3)	3 Memory (3,3)	3	VAX

- 第三章

- 五级流水线每一步的操作

1. Instruction fetch (IF)
2. Instruction decode and register operand fetch (ID/RF)
3. Execute/Evaluate memory address (EX/AG)
4. Memory operand fetch (MEM)
5. Store/writeback result (WB)

- 5级流水线的加速比是4，是因为触发器每一次前推都要花费一些时间
- 流水线注意点

- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Unbalanced lengths of pipe stages reduces speedup
- Time to "**fill**" pipeline and time to "**drain**" it reduces speedup

- Pipe stage or pipe segment
 - A decomposable unit of the fetch-decode-execute paradigm
- Pipeline depth
 - Number of stages in a pipeline
- Machine cycle
 - Clock cycle time
- Latch
 - Per phase/stage local information storage unit

- 平衡每个流水线的级数

Throughput =	$\frac{\text{Depth of the pipeline}}{\text{Time per instruction on unpipelined machine}}$
--------------	---

- 五级流水线步骤

Stage	ALU Instruction	Load/Store Instruction	Branch Instruction
IF	$IF/ID.IR \leftarrow Mem[PC];$ $IF/ID.NPC \leftarrow \text{if } ((EX/MEM.opc == \text{branch}) \ \& \ EX/MEM.cond) \ EX/MEM.AluOutput \text{ else } PC+4;$		
ID	$ID/EX.A \leftarrow Regs[IF/ID.IR[rs]]; \quad ID/EX.B \leftarrow Regs[IF/ID.IR[rt]];$ $ID/EX.NPC \leftarrow IF/ID.NPC; \quad ID/EX.IR \leftarrow IF/ID.IR;$ $ID/EX.Imm \leftarrow \text{sign-extend}(IF/ID.IR[\text{immediate field}])$		
EX	$EX/MEM.IR \leftarrow ID/EX.IR;$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.B \ \text{or}$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.Imm$	$EX/MEM.IR \leftarrow ID/EX.IR;$ $EX/MEM.ALUOutput \leftarrow ID/EX.A \ op \ ID/EX.Imm$	$EX/MEM.ALUOutput \leftarrow ID/EX.NPC + (ID/EX.Imm \ll 2)$ $EX/MEM.Cond \leftarrow ID/EX.A == 0)$
MEM	$MEM/WB.IR \leftarrow EX/MEM.IR$ $MEM/WB.ALUOutput \leftarrow EX/MEM.AluOutput$	$MEM/WB.IR \leftarrow EX/MEM.IR$ $MEM/WB.LMD \leftarrow Mem[EX/MEM.AluOutput]$ or $Mem[EX/MEM.AluOutput] \leftarrow EX/MEM.B$	
WB	$Regs[MEM/WB.IR[rd]] \leftarrow MEM/WB.AluOutput \ \text{or}$ $Regs[MEM/WB.IR[rt]] \leftarrow MEM/WB.AluOutput$	for load only: $Regs[MEM/WB.IR[rt]] \leftarrow MEM/WB.LMD$	

- IF

Instruction fetch (IF)

$IR \leftarrow Mem[PC];$

$NPC \leftarrow PC + 4$

- ID

- Instruction decode & register fetch (ID)

- $A \leftarrow \text{Regs}[\text{IR}_{6..10}];$

- $B \leftarrow \text{Regs}[\text{IR}_{11..15}];$

- $\text{Imm} \leftarrow ((\text{IR}_{16})^{16} \# \# \text{IR}_{16..31})$

- EX

- Execution & effective address (EX)

- Memory reference

- $\text{ALUOutput} \leftarrow A + \text{Imm}$

- Register - Register ALU instruction

- $\text{ALUOutput} \leftarrow A \text{ func } B$

- Register - Immediate ALU instruction

- $\text{ALUOutput} \leftarrow A \text{ op } \text{Imm}$

- Branch

- $\text{ALUOutput} \leftarrow \text{NPC} + \text{Imm}; \text{Cond} \leftarrow (A \text{ op } 0)$

- MEM

- Memory access & branch completion (MEM)

- Memory reference

- $\text{PC} \leftarrow \text{NPC}$

- $\text{LMD} \leftarrow \text{Mem}[\text{ALUOutput}]$ (load)

- $\text{Mem}[\text{ALUOutput}] \leftarrow B$ (store)

- Branch

- if (cond) $\text{PC} \leftarrow \text{ALUOutput}$; else $\text{PC} \leftarrow \text{NPC}$

- WB

- Write-back (WB)

- Register - register ALU instruction

- $\text{Regs}[\text{IR}_{16..20}] \leftarrow \text{ALUOutput}$

- Register - immediate ALU instruction

- $\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{ALUOutput}$

- Load instruction

- $\text{Regs}[\text{IR}_{11..15}] \leftarrow \text{LMD}$

- Clock cycle > clock skew (时钟脉冲相位差) + latch overhead

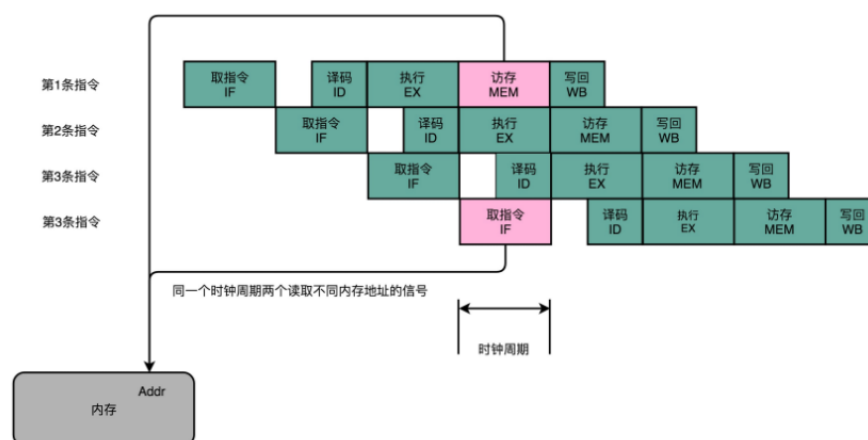
- HAZARD

- 三种类型

- 结构冒险

最典型的例子就是内存的数据访问。如下图：

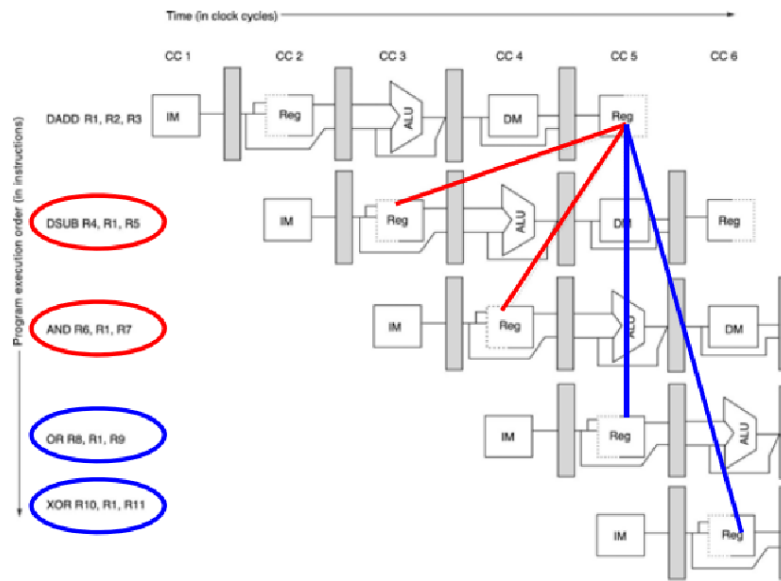
- 可以看到，在第1条指令执行到访存(MEM)阶段的时候，流水线里的第4条指令，在执行取指令(Fetch)的操作。
- 访存和取指令，都要进行内存数据的读取。
- 我们的内存，只有一个地址译码器的作为地址输入，那就只能在一个时钟周期里面读取一条数据，没办法同时执行第1条指令的读取内存数据和第4条指令的读取指令代码



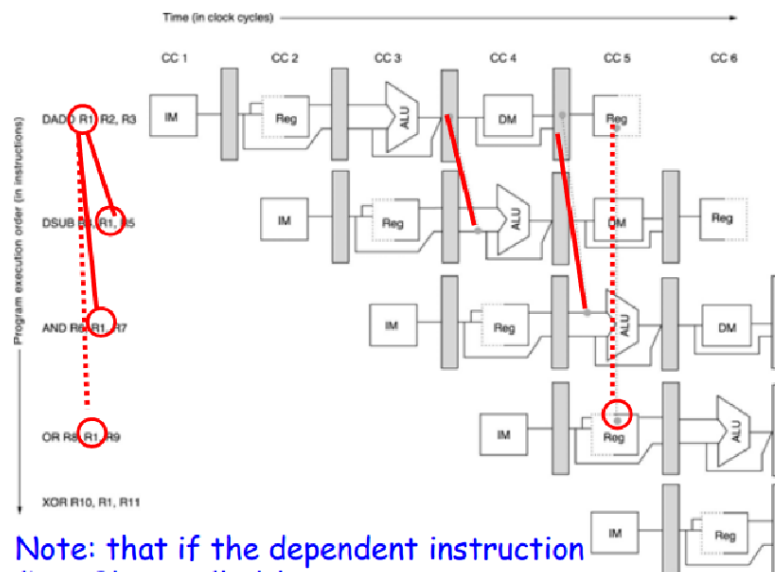
instruction	1	2	3	4	5	6	7	8	9
LW R10, 20(R1)	IF	ID	EX	MEM	WB				
SUB R11, R2, R3		IF	ID	EX	MEM	WB			
ADD R12, R3, R4			IF	ID	EX	MEM	WB		
STALL									
ADD R14, R5, R6					IF	ID	EX	MEM	WB

- **本质上是一个硬件层面的资源竞争问题，也就是一个硬件电路层面的问题**
- **（原因）CPU在同一个时钟周期，同时在运行两条计算机指令的不同阶段。但是这两个不同的阶段，可能会用到同样的硬件电路。**
- 当管道不能容纳某些指令组合时，就会发生结构危险
- （一些流水线处理器为数据和指令共享单一内存。因此，当一条指令包含一个数据内存引用(在MEM)时，它将与后面一条指令的指令引用(在IF)发生冲突)
- （原因）一个单元不是完全流水线□需要在服务下一个之前完成当前的指令，
- 后果：
 - 暂停管道，直到资源变得可用，这将建立一个bubble，从理想值增加CPI
- 解决方法
 - 完全避免结构危险是非常昂贵的：
 - 流水线化的功能单元成本更高(流水线化的FP ALU成本更高)
 - 资源重复(单独的指令和数据缓存意味着内存和带宽加倍)
 - 如果结构危险很少，就让它去吧。
 - 例如，如果FP操作很少，那么只需使用非流水线的FP ALU
- 数据冒险

- arise when an instruction depends on the result of a previous instruction in a way exposed by the pipeline overlapped execution
- 先写后读 (Read After Write, RAW)

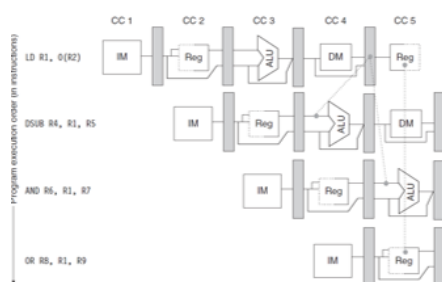


- 前推forwarding (by-passing)



Note: that if the dependent instruction (DSUB) is stalled (or an interrupt occurs in between the two instructions) then the forward path will not be activated

- 并不是所有的数据风险都可以通过转发来解决



A pipeline interlock (a hardware unit) detects a hazard and stalls the pipeline until the hazard is cleared.

LD R1, 0(R2)	IF	ID	EX	MEM	WB				
DSUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

- lw需要在WB才能用在下一个的EX
- 先读后写 (Write After Read, WAR)
- 写后再写 (Write After Write, WAW)
- 控制冒险

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles from branches}}$$

Pipeline stall cycles from branches = Branch frequency × Branch penalty

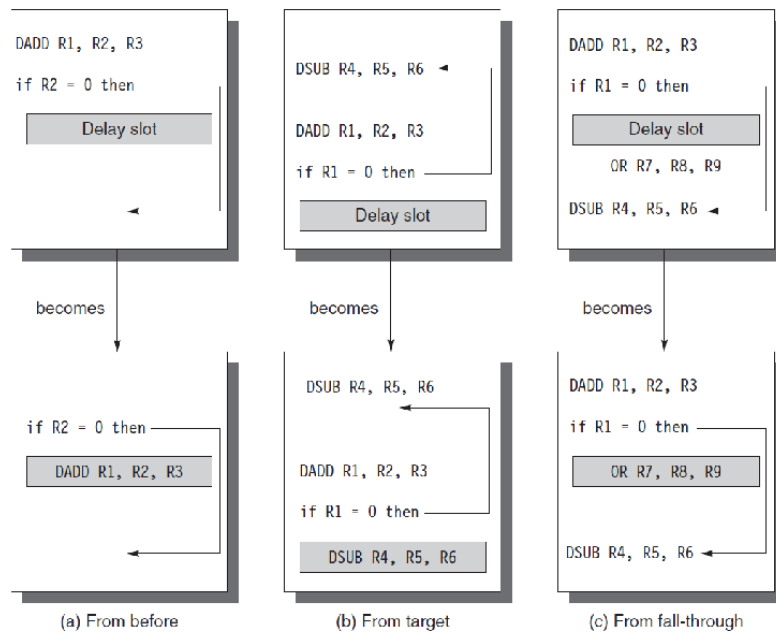
$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

- 产生于流水线的branch和jump
- 如果现在要执行哪条指令，是由之前指令的运行结果决定，而现在那条之前指令的结果还没产生，就导致了**控制冒险**。
- 解决方法

- ①冷冻或冲洗pipeline，像两个背靠背的IF
 - 编译器保存或删除分支后的任何指令，直到知道分支目标为止
 - 硬件/软件 简单，但惩罚是固定的，不能减少。
- ②猜Branch总是不被接受/失败

Untaken branch instruction	IF	ID	EX	MEM	WB		
Instruction $i + 1$		IF	ID	EX	MEM	WB	
Instruction $i + 2$			IF	ID	EX	MEM	WB
Instruction $i + 3$				IF	ID	EX	MEM
Instruction $i + 4$					IF	ID	EX
						MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB		
Instruction $i + 1$		IF	idle	idle	idle	idle	
Branch target			IF	ID	EX	MEM	WB
Branch target + 1				IF	ID	EX	MEM
Branch target + 2					IF	ID	EX
						MEM	WB

- 万一猜对了，照常进行。
- 如果Guess是错误的，因为指令是在ID阶段，而不是解码，将其更改为NOOP。
 - 从Branch指令的输出地址获取指令。
- ③使用延迟跳转 (delayed branch)



- 编译器将一条指令(或多条指令, 视需要而定)排在分支后面。指令总是被执行, 之后我们将知道Branch指令的ID结果。
- (a) 是最好的。
- (b)和(c)仅在(a)不可能的情况下使用。
- (b)如果Branch有较高的被取概率, 则优先考虑。
- (c)如果不是, 则优先考虑。
- 如果(b)和(c)的假设被证明是错误的, 它们都需要撤销它们的更改(例如寄存器)!
- 限制:
 - the restrictions on the instructions that are scheduled into the delay slots (对排入延迟槽的指令的限制)
 - our ability to predict at compile time whether a branch is likely to be taken or not (在编译时预测某个分支是否可能被执行的能力)
- 对于流水线实现来说, 控制风险比数据风险造成的性能损失更大。在第一个近似:
 - 根据分支频率的不同, 性能损失可能在10%到30%之间变化
 - 通常, 管道越深, 分支损失越严重(以时钟周期衡量)
- 后果: 强制使流水线暂停
 - CPI (无stall)

$$\begin{aligned} \text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}} \\ &= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}} \\ &= \frac{\text{CPI unpipelined}}{\text{CPI pipelined}} \times \frac{\text{Clock cycle unpipelined}}{\text{Clock cycle pipelined}} = \frac{\text{CPI unpipelined}}{\text{CPI pipelined}} \end{aligned}$$
 - CPI (有stall)

Speedup = $\frac{\text{CPI unpipelined}}{1 + \text{Pipeline stall cycles per instruction}}$

Speedup = $\frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$

SD	R1, 0(R2)		D	X	M	W					
DADDI	R2, R2, #4	不是存到R2, 所以下一跳不用stall	F	D	X	M	W				
DSUB	R4, R3, R2			F	s	s	D	X	M	W	
LD	R4, R3, R2			F	s	s	D	X	M	W	
BNEZ	R4, Loop				F	s	s	D	X	M	W
LD	R1, 0(R2)	BNEZ是在M阶段才会前推, 此时才知道跳	F	D							

LD	R1, 0(R2)	F	D	X	M	W
DADDI	R1, R1, #1	F	D	s	X	M

LD forwarding 结果只能从M到下一个X

technique	reduces
forwarding/ bypassing	potential data-hazard stalls
delayed branches	control-hazard stalls
basic dynamic scheduling (scoreboarding)	data-hazard stalls from true dependencies
dynamic scheduling with register renaming	data-hazard, anti-dep. & output dep. stalls
dynamic branch prediction	control stalls
issuing multiple instruction per clock cycle	ideal CPI
speculation	data-hazard and control-hazard stalls
dynamic memory disambiguation	data-hazard stalls with memory
loop unrolling	control hazard stalls
basic compiler pipeline scheduling	data-hazard stalls
compiler dependency analysis	ideal CPI, data-hazard stalls
software pipelining & trace scheduling	ideal CPI, data-hazard stalls
compiler speculation	ideal CPI, data-hazard stalls

- 按顺序解码和发出指令，但在指令的数据操作数可用时立即执行
- 它可能导致无序完成。
- 实现

- 我们需要将ID分成两个阶段
 - issue-decode 指令，检查结构冒险
 - 读操作数——一直等待直到不存在数据冒险，则读取操作数
- 在动态调度的pipeline中:
 - 按顺序发射指令
 - 在第二阶段(读操作数)，它们可能会停止或互相绕过，从而进入无序执行。
- 新的数据冒险

- 反依赖——WAR

between ADD.D and SUB.D (may lead to WAR hazard)

- 输出依赖——WAW

between MUL.D and ADD.D (may lead to WAW hazard)

• Scoreboard

- 目标：保持CPI $\approx$ 1
- 有序发射
- 无序执行
 - 原因：尽可能快的issue指令，关注issue和执行，包括所有的冒险检测
 - 检查结构冒险
 - 等待数据冒险的消失
 - 所有指令通过Scoreboard，并且Scoreboard记录所有的数据依赖关系。
 - 它可以确定指令何时可以读取其操作数并开始执行。
 - 如果一个指令必须暂停(RAW危险)，Scoreboard还可以找到它的操作数可用的最早时间。
 - Scoreboard还控制一条指令何时可以将其结果写入目标寄存器。
 - 避免WAW和WAR
- 相关名词

(1) 发射——如果指令的一个功能单元空闲，没有其他活动指令以同一寄存器为目标寄存器，则记分卡向功能单元发射指令，并更新其内部数据结构。这一步代替了 MIPS 流水线中 ID 步骤的一部分。只要确保没有其他活动功能单元希望将自己的结果写入目标寄存器，就能保证不会出现 WAW 冒险。如果存在结构性冒险或 WAW 冒险，则指令发射停顿，在清除这些冒险之前，不会再发射其他指令。当发射级停顿时，会导致指令提取与发射之间的缓冲区填满；如果缓冲区只是一项，则指令提取立即停顿。如果缓冲区是拥有多条指令的队列，则在队列填满后停顿。

(2) 读取操作数——记分卡监视源操作数的可用性。如果先前发射的活动指令都不再写入源操作数，而该源操作数可用。当源操作数可用时，记分卡告诉功能单元继续从寄存器读取操作数，并开始执行。记分卡在这一步动态解决 RAW 冒险，可以发送指令以进行乱序执行。这一步和发射步骤一起，完成了简单 MIPS 流水线中 ID 步骤的功能。

(3) 执行——功能单元接收到操作数后开始执行。结果准备就绪后，它通知记分卡已经完成执行。这一步代替了 MIPS 流水线中的 EX 步骤，在 MIPS 浮点流水线中耗用多个周期。

(4) 写结果——一旦记分卡知道功能单元已经完成执行，则检查 WAR 冒险，并在必要时停顿正在完成的指令。

• 组成部分

- 1、指令状态——指出该指令处于4个步骤中的哪一个
- 2、功能单元状态

(2) 功能单元状态——指出功能单元（FU）的状态。共有 9 个字段用来表示每个功能单元的状态。

- 忙——指示该单元是否繁忙。
- Op——在此单元中执行的运算（例如，加或减）。
- Fi——目标寄存器。
- Fj, Fk——源寄存器编号。
- Qj, Qk——生成源寄存器 Fj、Fk 的功能单元。
- Rj, Rk——指示 Fj、Fk 已准备就绪但尚未读取的标记。在读取操作数后将其设置为“否”。

- 3、寄存器结果状态

(3) 寄存器结果状态——如果一条活动指令以该寄存器为目标寄存器，则指出哪个功能单元将写入每个寄存器。只要没有向该寄存器写入的未完成指令，则将此字段设置为空。

• 示例

Scoreboard Example: Clock Cycle = 14

Instruction		Issue	Read Operands			Exec. Complete		Write Result	
L.D F6, 34(R2)		1	2			3		4	
L.D F2, 45(R3)		5	6			7		8	
MUL.D F0, F2, F4		6	9						
SUB.D F8, F6, F2		7	9			11		12	
DIV.D F10, F0, F6		8							
ADD.D F6, F8, F2		13	14						
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult-1	yes	MUL.D	F0	F2	F4			false	false
Mult-2	no								
Add	yes	ADD.D	F6	F8	F2			false	false
Divide	yes	DIV.D	F10	F0	F6	Mult-1		false	true
	F0	F2	F4	F6	F8	F10	...	F30	
FU	Mult-1			Add		Divide			

• addition operands are read and execution starts (will take two cycles)

• 循环展开

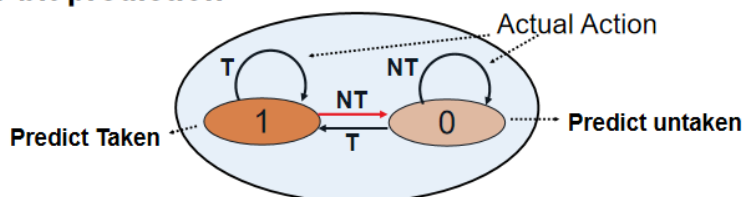
- 循环
- 归并

- 重命名
- 调度

• 第五章——动态分支指令预测

- 降低branch开销
 - 预测branch/jump指令和branch跳转方向 (taken or not taken)
 - 预测分支/跳转目标地址(对于taken 的分支)
 - Speculatively execute instructions along the predicted path
- 静态分支指令预测
 - 在运行前决定
 - always-not-taken
 - always-taken
 - backwards taken
 - forward not taken
 - profile-driven prediction
- 动态分支指令预测
 - 预测决策可能在程序执行期间发生变化
 - 1bit分支预测器

1-bit prediction

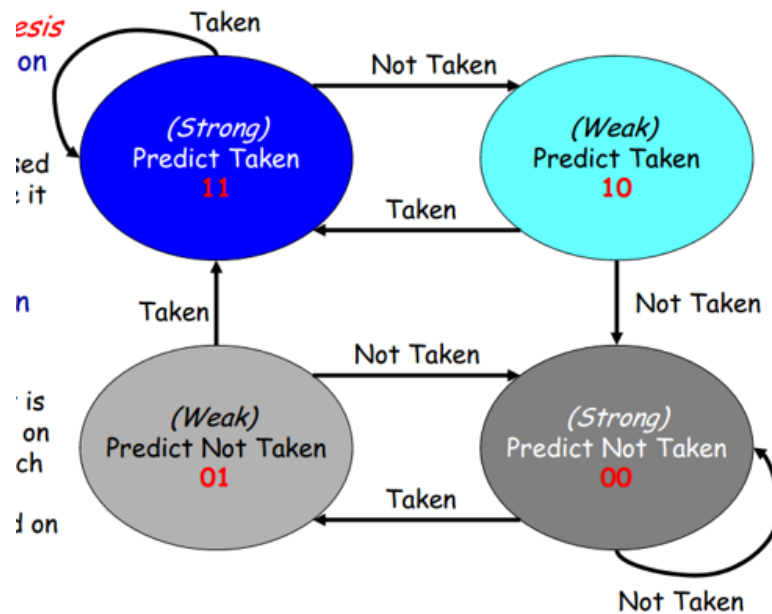


5

- 基本的分支预测：分支预测表（BHT）
 - 如果两个分支指令的地址部分是相同的，那么它们共享一个entry
 - 在循环嵌套中有缺点（错误预测两次，一次在进入时，一次在退出时，每次执行内部循环时。）
 - Consider a nested loop :

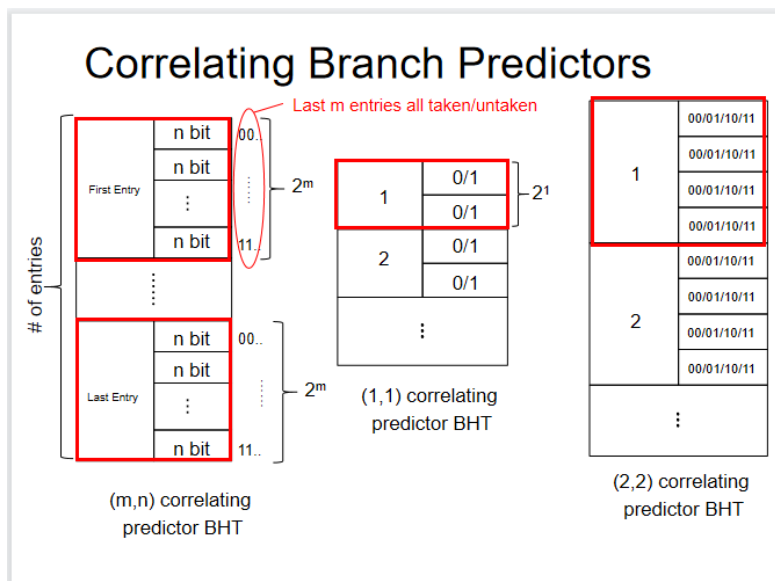

```

for (...) {
    for (i=0; i<10; i++)
        a[i] = a[i] * 2.0;
}
              
```
 - Mispredict twice, once on entry, once on exit, every time when the inner loop is executed.
- 2-bit预测方案（强到弱，弱到弱，弱到强/强到弱，弱到强；都可）



- 预测必须错两次才会改变
- 实现
 - 两位饱和寄存器
- 两位预测器方案仅适用单个branch的最近行为来预测该分支的未来行为，增加到更多位数不会有太大帮助

- Correlating Branch Predictors



- 示例

A Simple Example of (1,1) Predictor: Simulation - Cycle 1

```

BNEZ R1, L1
DADDIU R1, R0, #1
L1: DSUBUI R3, R1, #-1
    BNEZ R3, L2
    ...
L2: ...

```

```

if (d == 0)
    d = 1;
if (d == 1) {

```

Initial Status

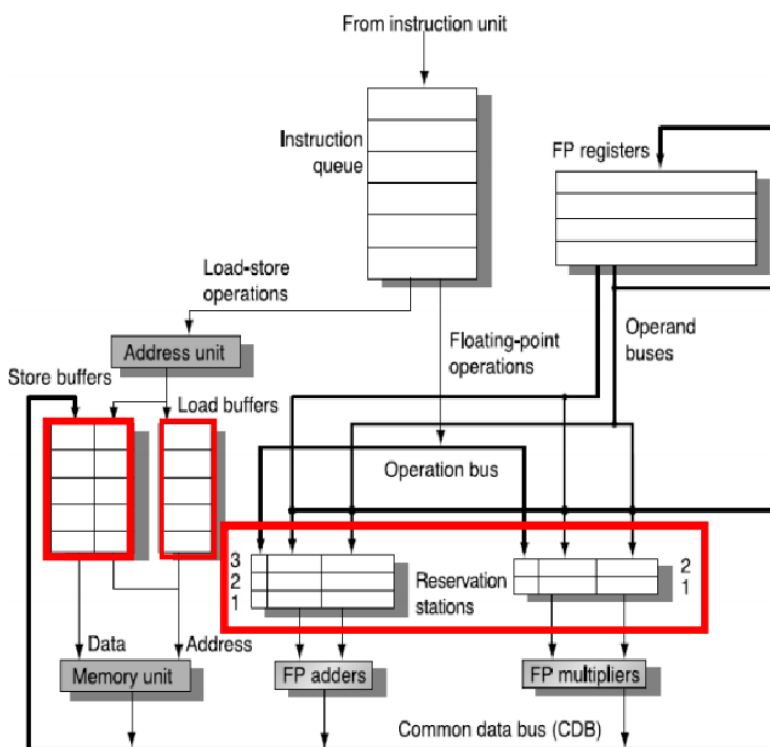
b1	NT	0	b1's last branch untaken
	NT	1	b1's last branch taken
b2	NT	0	b2's last branch untaken
	NT	1	b2's last branch taken

Assumption: d alternates between 2 and 0
X/Y: use X if last branch was not taken,
use Y if last branch was taken

d=?	b1 pred.	b1 action	b1 new pred.	b2 pred.	b2 action	b2 new pred.
2	NT/NT	T	T/NT	NT/NT	T	
0	T/NT	NT			NT	
2		T			T	
0		NT			NT	

A (1,1) predictor initialized to NT/NT mispredicts only at the first iter.

- 2 bit vs correlating
 - 2bit = (0,2) 关联
 - 2-bit Predictor with unlimited entries.
- 第六章——托马苏洛



- 检查寄存器文件中的操作数，如果可用，读取它们(寄存器重命名处理WAR和WAW冒险)
- 如果操作数不可用，则跟踪将产生这些操作数的功能单元
- Execution (EX)
 - 监视CDB以查找缺少的操作数
 - 当操作数在保留站可用时开始执行(RAW危险)
- Write Result (WB)
 - 使用CDB广播结果来注册文件和保留站(包括存储缓冲区写入内存)

• 名词解释

- op——源操作数的值
- Vj, Vk: 源操作数的值
 - 存储缓冲区有V字段，要存储的结果
- Qj, Qk: 产生源寄存器(要写入的值)的预留站
 - 注意:没有计分板上的准备好的flag; Qj Qk = 0 =>准备好了
 - 存储缓冲区只有Qi为RS产生的结果
- A: 保存加载和存储指令的内存地址
- Busy: 表示保留站/FU忙

• 注意点:

- 一瞬间就开始

MUL.D F0, F2, F4	3				• executing multiplication (still 8 cycles to go) • stalling division (waiting for multiplication) • issuing addition			
SUB.D F8, F6, F2	4	6						
DIV.D F10, F0, F6	5							
ADD.D F6, F8, F2	6							
Name	Busy	Op	Vj	Vk	Qj	Qk	A	
Load-1	no							
Load-2	no							
Add-1	yes	SUB.D	M[45(R3)]	M[34(R2)]				
Add-2	yes	ADD.D		M[45(R3)]	Add-1			
Add-3	no							
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]				
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1			
Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]				
Qi	Mult-1			Add-2	Add-1	Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 7

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3		
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

• committing subtraction, broadcasting result to ADD-2
• executing multiplication (still 7 cycles to go)
• stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	yes	ADD.D	res _(SUB.D)	M[45(R3)]			
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]	res _(SUB.D)			
Qi	Mult-1			Add-2		Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 9

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3		
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	

• executing multiplication (still 5 cycles to go)
• stalling division (waiting for multiplication)
• completed addition execution

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	yes	ADD.D	res _(SUB.D)	M[45(R3)]			
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]	res _(SUB.D)			
Qi	Mult-1			Add-2		Mult-2		

- tomasulo的主要优点
 - 具有分布的阻塞检测机制：如果多条指令都在等待一个结果，且指令的另一个操作数都已准备就绪，公共数据总线广播这个被等待的结果后，这些指令可以同时得到这个结果数据，并同时开始进入执行段。
 - 消除了数据的写后写和先读后写相关导致的阻塞
 - 基于cdb的旁路机制，通过广播从功能单位到保留站（CDB-based by-passing mechanism via broadcasting from functional units to reservation stations）
- 托马苏洛和scoreboard的对比

	Tomasulo's Algorithm	Scoreboard
introduced with	IBM 360/91 in 1967	CDC 6600 in 1964
resources	3 adders, 2 mul/div, 6 load, 3 store	7 int units, 4 FP units, 5 mem. reference
max window size	14	5
structural hazards	stall issuing	stall issuing
WAR hazards	avoided by register renaming	stall committing
WAW hazards	avoided by register renaming	stall issuing
control	based on reservation stations	scoreboard module
communication	broadcasting to reservation stations (and registers)	buses to/from registers

- With scoreboard both operands are read at once when they are both ready in the register file, while Tomasulo's algorithm reads each operand asap

	Tomasulo's Algorithm	Scoreboard
introduced with	IBM 360/91 in 1967	CDC 6600 in 1964
resources	3 adders, 2 mul/div, 6 load, 3 store	7 int units, 4 FP units, 5 mem. reference
max window size	14	5
structural hazards	stall issuing	stall issuing
WAR hazards	avoided by register renaming	stall committing
WAW hazards	avoided by register renaming	stall issuing
control	based on reservation stations	scoreboard module
communication	broadcasting to reservation stations (and registers)	buses to/from registers

- With scoreboard both operands are read at once when they are both ready in the register file, while Tomasulo's algorithm reads each operand asap

第七章——cache

Block

- unit of memory transferred across hierarchy levels

Set

- a group of blocks

Modern processors

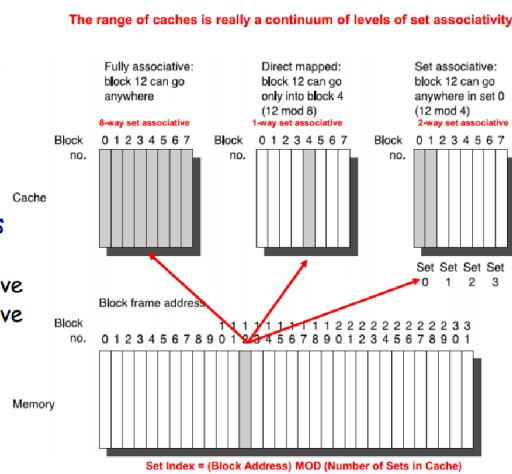
- direct map
- 2-way set associative
- 4-way set associative

Modern memories

- millions of blocks

Modern caches

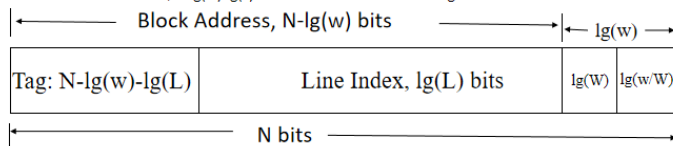
- thousands of block frames



直接映射

Direct Mapping

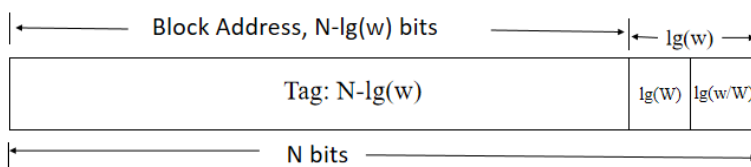
- Each block of main memory maps to only one cache line.
- Assume total N -bit memory address. It can address 2^N bytes.
- Assume each block has w bytes. Upon a miss, a block (total w bytes of data) is fetched from memory.
 - Alternatively, you can say each block has W words, $w = 4W$.
 - So byte address (or offset) within a block is $\lg(w)$ bits, and word address (or offset) is $\lg(W)$ bits.
 - So the number of blocks in main memory is $2^N/w$, hence the block address is $\lg(2^N/w) = N - \lg(w)$.
- Assume the cache can host L lines/blocks. So the line's index is $\lg(L)$ bits.
- When we place a block to a cache line, the block address is split into two parts:
 - The LSB of $\lg(L)$ bits in block address tells the cache line the block maps to.
 - The rest MSB bits, $N - \lg(w) - \lg(L)$ in block address becomes a tag.



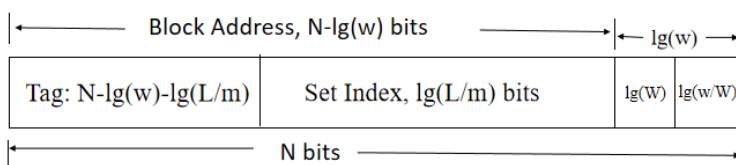
- # Bits in a block = # Tag Bits + # Data Bits + Valid bit**

- 注意：一个block中的bit只包含tag、data、valid
- 优点及缺点
 - 简单、便宜
 - 给定块的固定位置
 - 缺点：如果一个程序重复访问两个映射到同一行的块，缓存丢失是非常高的
- 全相连（没有index）

- Each block of main memory maps to any cache line. CPU chooses which line to map
- Assume total N -bit memory address, can address 2^N bytes.
- Assume each block has w bytes. Upon a miss, a block (total w bytes of data) is fetched from memory.
 - Alternatively, you can say each block has W words, $w = 4W$.
 - So byte address (or offset) within a block is $\lg(w)$ bits, and word address (or offset) is $\lg(W)$ bits.
 - So the number of blocks in main memory is $2^N/w$, hence the block address is $\lg(2^N/w) = N - \lg(w)$.
- When we place a block to a cache line, the whole block address is used as its tag.
 - The whole block address, which is $N - \lg(w)$ bits, becomes the tag.



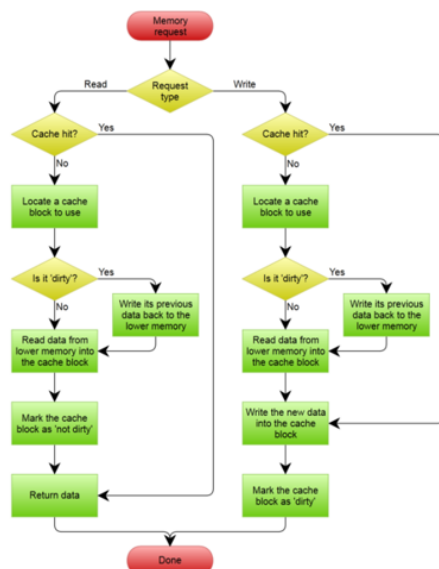
- 优点
 - 在确定cache中缓存的位置时很灵活
 - 贵
- 组相连



- 两个写策略

- 写回 (写分配)

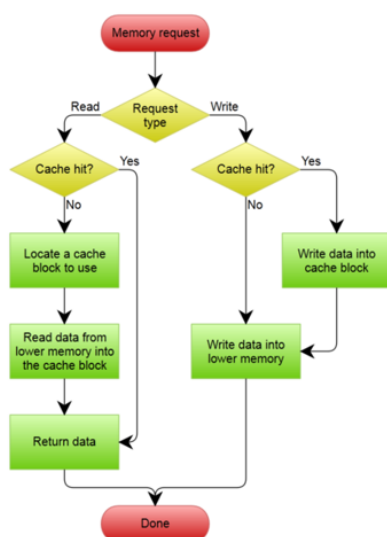
A Write-Back cache with Write Allocation



- 当cache发生缺失后，仅更新cache，先不更新主存，并设置一个脏位
 - 使用更少的内存带宽，是一种低功耗的方法
- 优点：
 - 写入操作以写cache的速度进行的
 - 一个块内多次写操作只需要一次对底层内存的写操作
 - 一些写操作不会进入内存
 - 回写使用更少的内存带宽(适用于多处理器)和更少的耗电量(适用于嵌入式应用)

- 写直达 (不写分配)

A Write-Through cache with No-Write Allocation



- 同时在cache 和 memory上更新
 - 需要一个写缓冲区来最小化写暂停
- 优点：
 - 比写回更容易实现
 - 缓存总是clean，所以更易miss

- 主存有最新的数据副本
- 数据一致性好

- 对比

Write-Through	Write-Back
Simpler control logic	More complex control logic
More predictable timing simplifies processor control logic	More variable timing
Easier to make reliable	Harder to make reliable, sometimes cache has only copy of data
More write traffic	Less write traffic

- 写分配

- 只在数据写不命中产生作用，即，给数据分配一个cache line。先在主存块中更新到主存中，然后分配一个cache行，将数据写到Cache中
 - 这种方式充分利用了空间局部性，但每次写不命中都要从主存读一个块到Cache中，增加了读主存的开销。

- 不写分配

- 直接把数据写回主存而不加载数据到缓存
- 这种方式可以减少读主存的时间，没有利用好空间局部性。

以上内容整理于 [幕布文档](#)